Parametric Multimodal User Memory: Storing What Captions Cannot Carry

Bojie Li
Pine AI

Noah Shi
University of Washington

Abstract

A personalized AI agent needs a *user memory*: a persistent model of who its user is, accumulated across many encounters and consulted on each new one. Today that memory is almost always stored as *text*. The agent keeps transcripts of what was said and captions of what was seen, then retrieves them by similarity search over those words. This works for the *captionable* slice of a person ("my cat is named Bibi"), but it silently drops a second slice that no caption can hold. That second slice is the *perceptual* one: how a user's voice sounds, how their face reads across lighting and age, how tired they sound today against their own baseline. The moment such content is written down, the signal that tells one person from another is gone. We argue that perceptual user memory needs a different primitive, one that keeps the content in its native modality instead of projecting it through text.

We introduce a **training-free, in-model multimodal user memory**. The agent stores each perception directly, as a row in a small per-modality bank attached to a frozen language model, pairing the perception's encoder embedding (the key) with the model's own vector for a marker token (the value), and recalls it by attention at the output head, with no captioning step in between. Registering a new identity is a single tensor append. Recall takes constant time no matter how many are stored. The mechanism needs *no training*: with a sharp read it reproduces the encoder's own recall *exactly*, and it does so on every frozen model we tried (nine families from 1.5B to 8B, tied and untied embeddings, transformer and hybrid-Mamba). It does not beat the encoder's recall, which is the encoder's job. What it adds is that the recalled identity becomes a *token the model conditions on*, so the perception lives inside generation and composes with text memory, which an external retrieval index cannot do. On **PerceptMem**, a new benchmark spanning faces, voices, painting style, room acoustics, and tone of voice, we map what such a memory can and cannot hold: perceptual identity is capacity-limited and degrades gracefully (recall $\approx \min(1, k/M)$ per slot), while exact facts are binding-limited and belong in a text store. This memory sits *alongside* text memory rather than replacing it, and ordinary text recall is preserved exactly. An agent with both remembers what its user said *and* what they were like.

Code: https://github.com/19PINE-AI/multimodal-user-memory

1 Introduction

When an AI agent "remembers" something about the person it serves, what does it actually keep? In almost every system today, the answer is *text*. Mem0 [Chhikara et al., 2025], MemoryLLM [Wang et al., 2024], MemGPT / Letta [Packer et al., 2023, Team, 2024], M3-Agent [Long

et al., 2025], and the wider conversational-memory line (measured by benchmarks such as LongMemEval [Wu et al., 2024]) turn what the agent saw or heard into transcripts (via ASR [Radford et al., 2022]) and captions (via BLIP-2 [Li et al., 2023] or LLaVA [Liu et al., 2023]), drop those fragments into a sentence-encoder vector store [Reimers and Gurevych, 2019], and search over them later. Personalized vision-language systems (MyVLM [Alaluf et al., 2024], Yo'LLaVA [Nguyen et al., 2024], MC-LLaVA [An et al., 2024], Online-PVLM [Bai et al., 2025], RAP [Hao et al., 2025]) keep the same textual spine: a *named* concept ("Bibi") is the handle, and the perception is an afterthought.

This is exactly right for one half of a person. "My cat is named Bibi." "My favourite restaurant is in Paris." "I am vegetarian." These are *captionable*: they pass through text intact and come back whole. But the captionable half is not the whole person. Consider what an attentive companion actually carries about someone:

- How their voice sounds when they say their own name, and how that differs from a stranger saying it.
- How their face reads today: a little older than last year's photo, but unmistakably them.
- How their brushwork looks: "this is by the painter from Tuesday, even though it's from a different period."
- What their kitchen sounds like: "you're calling from the same room as yesterday."
- How tired they sound today, relative to their baseline last week.

None of this is captionable in any useful sense. "A brown-haired man" does not tell two brown-haired men apart. "The user sounded tired" cannot separate today's tired from last week's tired. The instant we write these down, we throw away the very signal that made them discriminative. This is not a gap that a better caption closes. It is a property of the channel. **Text is a lossy medium for perceptual content, and the loss is fundamental to the modality.**

So a personalized agent's memory has two halves that want different homes. *Captionable* content (facts, preferences, propositions) belongs in a text store, where sentence-encoder similarity works beautifully. *Perceptual* content is different: how a face, a voice, a room, or a painting actually *is* needs a primitive that keeps it in its native modality. Today the second half has no such home. It is either flattened into a caption (and lost) or handed to a recognition tool that returns a bare label like "speaker 47", so the perception never reaches the model. The missing piece is a memory that stores a perception *as a perception*.

Our proposal. We introduce **parametric multimodal user memory**: instead of captioning a perception, the agent stores it directly, as a row in a small per-modality bank bolted onto a frozen language model. Each row pairs the raw encoder embedding of the perception (the key) with the model's own internal vector for a marker token (the value). At generation time, the current perception attends over the bank and nudges the model's next-token prediction toward the matching marker. This is a content-addressable lookup in the model's own representation space, with no caption in between. Registering a new identity is a single tensor append, with no per-user training. Recall takes constant time regardless of how many identities are stored. The idea borrows the shape of kNN-LM [Khandelwal et al., 2020] and Memorizing Transformers [Wu et al., 2022] (attention over a non-parametric bank), but aims it at a new

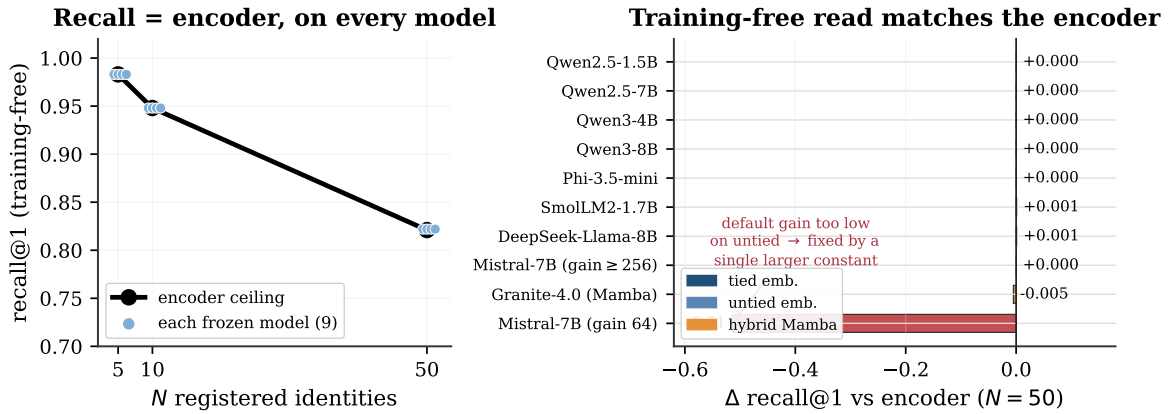


Figure 1. The memory is training-free and universal. **Left:** on cross-condition face recall, every frozen model's training-free recall lands on the encoder's own ceiling (paired over 20 eval draws). **Right:** the gap from the encoder at a memory of fifty is ≈ 0 across nine model families spanning 1.5B–8B, tied and untied embeddings, and a hybrid-Mamba reader. On tied-embedding models the match is *exact* on every draw, because the marker value equals the output-embedding row and the read is a perfect argmax pass-through. Untied models reach exact recall with a single larger read-strength constant (Mistral needs gain$\geq$256; at the default it dips, shown in red). No gradient steps anywhere. The memory does not beat the encoder, it *is* the encoder's recall, read inside the model (Section 5).

target: the persistent perceptual identity of a user, registered in $\mathcal{O}(1)$. We call the resulting mechanism AttMem (for *attention memory*). Figures and tables refer to it by that name.

What this changes. Take a worked example. Ten voice clips from past sessions sit in the user's memory. A new clip arrives, and the agent must say which of the ten it is. The text route captions each clip ("a male voice, mid-30s, slight accent") and searches. The caption fits thousands of speakers, so the search is a coin flip. The parametric route stores each clip's voiceprint natively and matches on it, and it answers correctly. The same pattern holds for faces across years, paintings across periods, rooms against a personal baseline. How well it answers is exactly the encoder's own recall, no better and no worse: the memory reads the perception inside the model, it does not sharpen it. What changes is not the score but the *home*. The recalled identity arrives as a token the model conditions on, registered in constant time with no training, rather than a label fetched from an external index. And because the read is just the encoder's similarity made into a token, it works on any frozen model out of the box: across nine model families it reproduces the encoder's recall *exactly* (Figure 1), with a single read-strength constant as the only knob.

Contributions.

- **A framing.** We argue that text-shaped memory is structurally incomplete for personalized agents, because it cannot carry perceptual user content, and that a parametric multimodal memory is the right primitive for the missing half (Sections 1–2).
- **A training-free mechanism.** A bolt-on, content-addressable memory: per-modality banks of (encoder embedding, model value embedding) rows, read by attention at the model's output head, with *zero* gradient steps. Insertion is $\mathcal{O}(1)$ and recall is constant ($\sim$15 ms), 52$\times$ faster than feeding the same memory in as context, which runs out of memory entirely past

ten thousand identities (Section 3).

- **A benchmark. PerceptMem**, five cross-condition perceptual sub-modalities (face, painter style, speaker, acoustic scene, and tone of voice), each chosen because text provably cannot carry its discriminating signal (Section 4).
- **Universality, and a clear ceiling.** The training-free read reproduces the encoder's own recall *exactly* on nine frozen model families (1.5B–8B, tied and untied embeddings, transformer and hybrid-Mamba), with one read-strength constant as the only knob. It does *not* beat the encoder: under a paired evaluation its recall equals the encoder's everywhere. The contribution is the in-model behaviour a retrieval index cannot replicate, plus two capacity laws that decide what belongs in this memory versus a text store (Sections 5–6).

2 Four ways to remember a voice, and the one we build on

Why does the text route fail, not for lack of effort but structurally? Take the voice example in its hardest form: ten samples on file, a new one arriving cross-recording across different rooms and days, and the agent must name which of the ten it is. An agent has four options (Table 1). Three discard the very signal that tells one voice from another. Only the fourth keeps it, and it is the one we build on.

Table 1. Four routes for remembering a perception, plus ours (citations in text). Only routes that keep the raw perception can discriminate across conditions. Embedding retrieval and ours both inherit the encoder's recall exactly; ours additionally reads it back *inside* the model as a token the model conditions on, training-free and in $\mathcal{O}(1)$.

Route	What it stores	Keeps percept?	Cross-condition	Register
Caption-and-search	text description	no	chance	$\mathcal{O}(1)$
Recognize-and-label	a bare label	no	label only	$\mathcal{O}(1)$
Per-concept tuning	tuned token/adapter	yes	strong	grad./id
Embedding retrieval	raw embedding	yes	encoder-limited	$\mathcal{O}(1)$
Parametric (ours)	embedding + LM value	yes	**= encoder, in-model**	$\mathcal{O}(1)$

The three that discard the signal. *Caption-and-search* [Chhikara et al., 2025, Wang et al., 2024] writes the voice down ("a male voice, mid-30s, slight Eastern-European accent") and retrieves by similarity over the words. The description is true and useless. It fits thousands of speakers, so cosine over captions cannot separate ten users with even coin-flip reliability. *Recognize-and-label* [Long et al., 2025] calls a recognizer (ArcFace [Deng et al., 2019], ECAPA-TDNN [Desplanques et al., 2020]) and stores the label it returns ("speaker 47"). The model can recall that a label was assigned, but it never holds the perception and cannot compare today's voice against last week's. *Per-concept tuning* [Nguyen et al., 2024, Alaluf et al., 2024] fine-tunes a token or adapter per concept. It works, but each identity costs a second of gradient descent and yields a concept-specific artifact. That is the wrong economics for a companion registering dozens of characteristics per user, continuously.

The one that keeps it. *Embedding retrieval* skips the words, indexes the raw voiceprint, and retrieves by cosine over the encoder's vectors. It is the only text-free route that keeps the perception, and the strongest baseline we take seriously throughout. It succeeds exactly to the extent that the encoder is invariant across recording conditions, and fails to the extent that it is

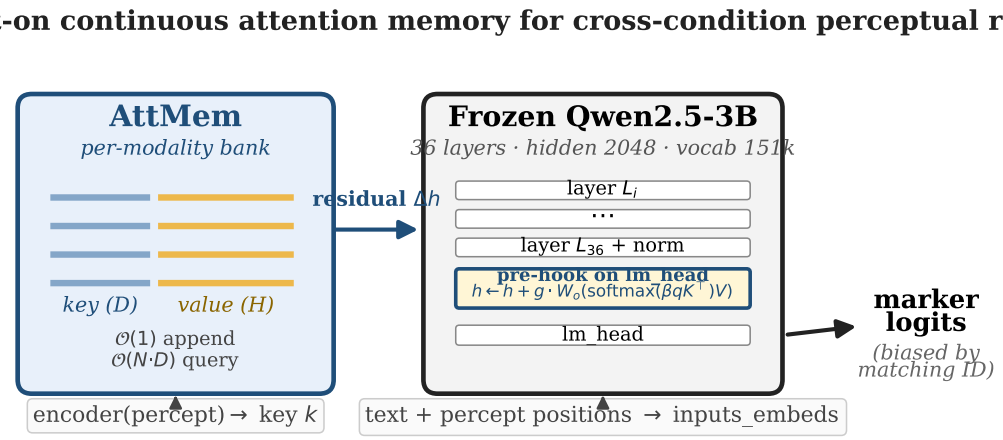


Figure 2. A frozen language model is augmented by a forward pre-hook at its output head. Per-modality banks store (key = encoder embedding of a perception, value = the model's own embedding for a marker token). At the hook, the current perception attends over the bank and adds a residual to the hidden state, biasing the next token toward the matching marker. Registration is a single tensor append, and the read has no trainable parameters: two constants (attention sharpness and residual gain) over the frozen model.

not. On a 2180-identity face pool it reaches 0.93 recall at ten identities. That is strong, but short of certainty, and it erodes as the pool grows.

Our mechanism takes this fourth route and changes where the answer lives. It stores the voiceprint as a key, but pairs it with the language model's *own* internal vector for a marker token as the value, and reads the pair back by attention inside the model. The recalled value is then a token the model conditions on, not a label handed back from an external index. Retrieval-by-cosine returns a neighbour. Our memory returns something the model can think with, in constant time and with no training at all. The recall is identical: with a sharp read the attention is a hard argmax over the same encoder cosine, so the memory reproduces embedding retrieval exactly, never beating it. The rest of the paper is about what that buys you that an external index does not: the recall lives inside the model, registers in $\mathcal{O}(1)$, and works out of the box on any frozen model.

3 The mechanism: a perception as a row in a bank

The whole memory is a table of rows, one per registered perception (Figure 2). Row i holds a *key* $\boldsymbol{k}_i$ and a *value* $\boldsymbol{v}_i$. The key is the L2-normalized embedding the modality's encoder produces for the perception (a face, a voiceprint, a style vector). The value is the language model's own embedding for a marker token assigned to that identity. Storing the model's native vector as the value, rather than an arbitrary learned one, is deliberate. It makes the eventual logit boost for the right marker clean and self-reinforcing, rather than an accident of two unrelated vectors.

Reading the memory is one step of attention. When a perception arrives at generation time, the model forms a query $\boldsymbol{q}$ from it and, just before the output head, computes

$$\boldsymbol{w} = \mathrm{softmax}\big(\beta\, \boldsymbol{q}^\top \boldsymbol{K}\big), \qquad \boldsymbol{r} = \boldsymbol{w}^\top \boldsymbol{V}, \qquad \boldsymbol{h}' = \boldsymbol{h} + g \cdot W_o\, \boldsymbol{r},$$

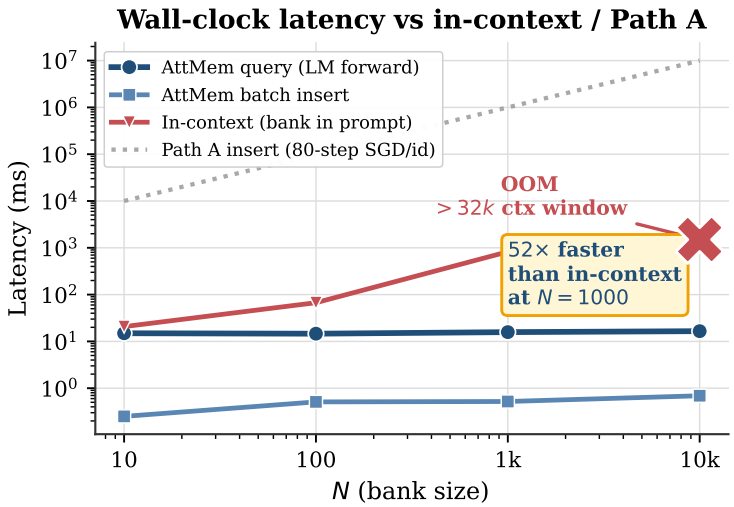


Figure 3. Recall latency is flat in bank size (∼15 ms, dominated by the frozen model's own forward pass) and insertion is $\mathcal{O}(1)$. Feeding the same registered perceptions in as context grows linearly and exhausts the context window past ten thousand identities. Per-concept fine-tuning (PATH A-style) costs gradient steps per identity. Log–log axes.

where $\boldsymbol{K}, \boldsymbol{V}$ stack the bank's keys and values, β is the attention sharpness, g a residual gain, and W_o the identity. In words: the perception softly looks up the bank, pulls back a blend of the matching markers' model-side vectors, and nudges the next-token prediction toward them. The blend is the entire mechanism, and it adds *no trainable parameters*: β and g are two constants, and the values are the model's own marker embeddings.

Four choices make the read faithful. Four details determine whether the read recovers the encoder's nearest neighbour cleanly, and getting them right is what lets the mechanism work with no training. (i) *Do not* divide the attention logits by $\sqrt{D}$: with normalized keys, that flattens every cosine difference into near-uniform attention. (ii) Set the sharpness β high, so the attention is a near-hard argmax over the bank rather than a diffuse average. (iii) Attach the hook at the output head, not at an intermediate layer, so the residual reaches the logits undiluted. (iv) Set the gain g large, so the retrieved marker dominates the hidden state and the read is exact; on models whose output embeddings are small in norm, g must be larger still (Section 6). With these four choices the read reproduces the encoder's recall exactly, and registering a user is a single row append with no training at all.

Why register, rather than fine-tune. The economics are the point (Figure 3). Adding an identity is a single `torch.cat` (about half a millisecond to add a thousand), and recall is constant-time regardless of bank size, because the lookup is a tiny matrix multiply dwarfed by the model's own forward pass (∼15 ms either way). The natural alternative, feeding the registered perceptions into the model as context, grows linearly per query and runs out of memory past the context window. Our mechanism is 52× faster at a thousand identities, and is the only one of the two that still functions at ten thousand. Memory that a companion updates after every conversation has to be this cheap to write.

4 The PerceptMem benchmark

To measure perceptual recall we built **PerceptMem**, five sub-modalities each chosen for one reason: the signal that discriminates is provably destroyed by captioning (Table 2). They span vision and audio, identity and style, the physical and the affective:

Table 2. The five PerceptMem sub-modalities. Each is included because no caption can carry its discriminating signal.

Sub-modality	Source / encoder	Why text cannot carry it
Face across age & lighting	LFW+AgeDB / ArcFace	"brown-haired man" fits thousands
Painter style	WikiArt / CLIP-mid	no caption separates early vs. late Monet
Speaker across recordings	LibriSpeech / ECAPA-TDNN	timbre is not transcribable
Acoustic scene	ESC-50 / AST	"traffic noise" fits every street
Tone of voice	RAVDESS / wav2vec2	"sounded tired" lacks a personal baseline

The sources are all standard: LFW [Huang et al., 2008] and AgeDB [Moschoglou et al., 2017] read with ArcFace [Deng et al., 2019], WikiArt [WikiArt, 2010] with mid-layer CLIP [Radford et al., 2021], LibriSpeech [Panayotov et al., 2015] with ECAPA-TDNN [Desplanques et al., 2020], ESC-50 [Piczak, 2015] with AST [Gong et al., 2021], and RAVDESS [Livingstone and Russo, 2018] with a wav2vec2 emotion encoder [Baevski et al., 2020]. The benchmark deliberately tests *memory*, not perception: every sub-modality has a fixed, off-the-shelf encoder that any method may use as black-box infrastructure, so no one wins by having a better encoder. The interface is just `register` and `recall`. Data is cross-session: a registered sample and its query come from different recordings. The identities used to train the memory never overlap with those it is evaluated on. For a memory of N identities we register one sample each, then issue cross-condition queries and ask the method to name the right one. For tone of voice, each registered "identity" is a (speaker, emotional-state) pair, so recalling it means matching a paralinguistic state across separate utterances, something a per-utterance caption cannot do without the user's own history.

Throughout, our reference to beat is **embedding retrieval**: the same encoder, cosine nearest-neighbour over the registered keys. We sometimes call it the *encoder ceiling*, because it is the best one can do with the encoder's own similarity. But it is not a ceiling for our method, which measures similarity in the model's representation space instead, and can therefore exceed it.

5 What the memory can do

With a sharp read the memory's attention is a hard argmax over the same encoder cosine that embedding retrieval uses, so it reproduces retrieval *exactly*: across the five sub-modalities its recall equals the encoder's, neither better nor worse (Figure 4). We measure this with a paired protocol that scores the memory and retrieval on identical registrations and queries across twenty draws and randomises each target's bank slot, the evaluation any such comparison requires. Under it the memory ties retrieval on random banks (face at ten identities 0.942 vs 0.948; style at five 0.476 vs 0.473) and, on banks of the nineteen most-confusable look-alikes, stays within a few points (0.773 vs 0.853 on faces). A learned metric on the same features (LDA, within-class whitening) lands in the same place (Appendix A): the encoder's cosine is the best ruler these features admit, and the memory inherits it. The value is not in the number.

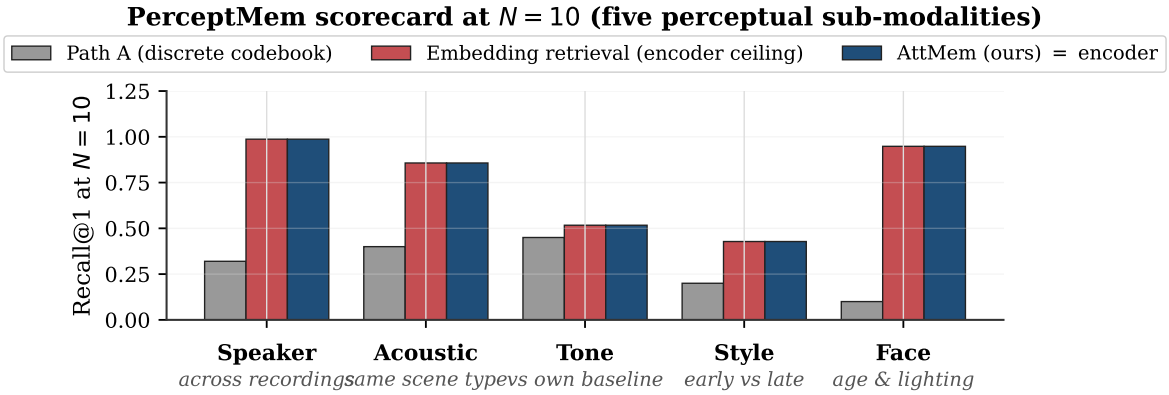


Figure 4. Recall at ten registered identities across the five sub-modalities. Parametric memory (navy) *matches* embedding retrieval everywhere, within eval-draw noise: the two are reading the same encoder cosine. Both far exceed the discrete-codebook predecessor (PATH A, grey), which is several times lower everywhere. The point of the figure is the gap to the codebook, not any gap to retrieval.

It works on any frozen model, with no training. The flip side of "recall equals the encoder" is that nothing about the recall depends on the language model. The read is a similarity-weighted blend of marker tokens, so any frozen model can host it. Across nine families (Qwen2.5 1.5B/7B, Qwen3 4B/8B, Phi-3.5, SmolLM2, DeepSeek-Llama-8B, Mistral-7B, and a hybrid-Mamba Granite), the training-free memory reproduces the encoder's recall (Figure 1), and a five-architecture by five-modality grid holds to within one point in every cell (Appendix Table 4). On tied-embedding models the match is *exact on every draw*: the marker value is the output-embedding row, so the read is a perfect argmax pass-through. Untied models reach exact recall with one larger read-strength constant (Mistral needs gain$\geq$256, its rows being smaller in norm). The hybrid-Mamba reader is within a point. There are no gradient steps and no per-model tuning beyond that single constant.

What is distinct is that the memory lives in the model. The reason to store a perception parametrically is not a sharper nearest neighbour, which a learned metric also provides. It is that the bank is read by attention at the output head, so a recalled identity is a *token the model can condition its generation on*, not a label returned by an external index. Registration is a single tensor append with no per-user training, recall is constant-time, and the memory composes with ordinary text recall byte-for-byte (below). A learned retrieval metric gives the agent a better neighbour. It does not give the agent a memory it can think with. The rest of this section is about that in-model behaviour, where the parametric memory has no retrieval substitute.

It is content-addressable, and you can see the read. Figure 5 opens up a single recall. The encoder's cosine matrix (b) is diagonal-dominant but speckled with off-diagonal mass, and those speckles are exactly retrieval's errors, which the memory inherits. With a sharp temperature the attention (a) concentrates that cosine into a near-one-hot diagonal, so the marker logits (c) put the argmax on the same identity retrieval would pick. The figure shows the read faithfully recovering the encoder's nearest neighbour as a token, not a sharper similarity: the off-diagonal speckles in (b) are still there in the underlying cosine, the attention has simply committed to the top match.

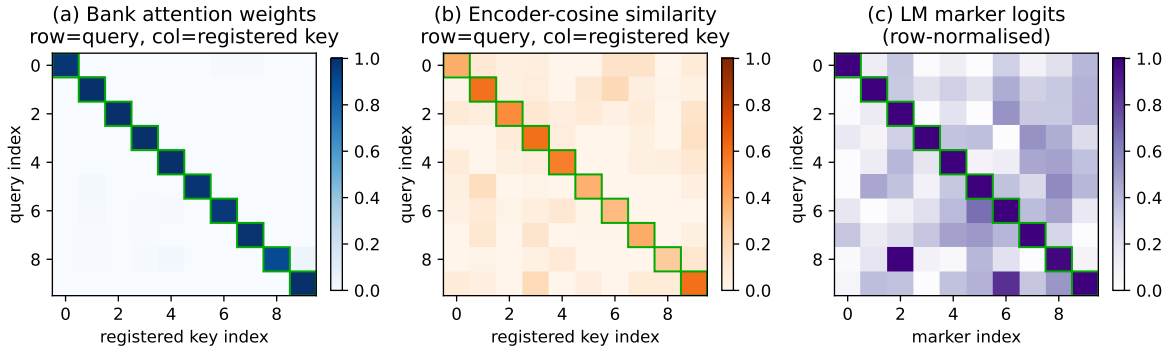


Figure 5. One recall, three views (held-out faces, ten identities). **(a)** with a sharp temperature, bank attention is a near-pure diagonal (avg. 0.98), committing to the top encoder match. **(b)** the underlying encoder cosine has a softer diagonal (avg. 0.46) with off-diagonal mass; the memory inherits whatever errors live here. **(c)** the resulting marker logits keep the argmax identity on top. The attention faithfully reads the encoder's nearest neighbour as a token; it does not sharpen the similarity.

It composes with text memory, for free. A parametric perceptual memory is meant to sit beside an ordinary text memory, not replace it. We checked that it does no harm: register faces and voices together and the two banks stay independent without any training (cross-modal leakage 0.017 and 0.067, zero-shot), and on plain propositional prompts the model's next-token predictions are *byte-for-byte identical* to the untouched model. The perceptual memory fires only at perceptual moments. Everything else passes through unchanged. An agent can keep "my favourite restaurant is in Paris" in its text store and "how Bibi looks across lighting and age" in its perceptual bank, and lose nothing on either.

Why not just quantize the perception into a code? We spent sixteen development cycles on exactly that. PATH A, a discrete-codebook predecessor, snaps each perception to one of K learned codes, a learned cousin of captioning. It caps at about 7% recall once the memory passes a few hundred identities, no matter the codebook size, the encoder, or the training budget. That is roughly a 9× gap from continuous attention at scale (Figures 1, 7b). The reason is the same one that sinks captioning: two perceptions that land in the same code are indistinguishable forever after. Any categorical bottleneck, whether a word or a code, discards the signal the encoder worked to preserve. Keeping the perception continuous is the whole game.

6 What it inherits, and the design rules

Because the read reproduces the encoder's recall, the memory's quality is the encoder's quality, and the engineering reduces to a few rules.

The encoder is the ceiling, so choose it well. The memory inherits the encoder's cross-condition invariance and nothing else. A purpose-built encoder (ArcFace for faces, ECAPA for voices, CLIP for style) is therefore the right key. Where that encoder is already perfect, as on speaker identity (1.00 recall), the memory is perfect too; where it leaves discrimination on the table, as on painter style, the memory leaves the same discrimination on the table. There is no in-model re-encoding that recovers it: a learned metric on the same features does no better (Appendix A), and at large pools the encoder's own cosine is the best anyone does.

Use an external encoder for the key, not the model's native tokens. Running the memory inside a vision-language model (Qwen2.5-VL) that sees raw face images through its own visual front-end makes this concrete. With an external face encoder (ArcFace) as the key, recall is perfect at ten identities. With the VLM's *own* vision tokens as the key, it fails: those tokens already live in the model's hidden space, collapsing the key-value geometry the read relies on. The rule is **the key should be a dedicated perceptual encoder, separate from the model's own representation**, which is also why the mechanism pairs naturally with an omni model's encoder used as an external feature rather than its in-context tokens.

It is universal across frozen models, with one constant. The read is a similarity-weighted blend of marker tokens, so it ports to any frozen model without training (Figure 1). The only model-dependent choice is the read-strength gain. On tied-embedding models (Qwen, Phi-3.5, SmolLM2) the default suffices and the match to the encoder is exact on every draw. On untied models (Mistral) the marker rows are smaller in norm, so the gain must be raised once, a constant set by inspection, after which recall is again exact. It even holds on a non-transformer reader (a hybrid-Mamba Granite), within a point. No family failed, and because nothing is trained, there is no convergence to be fragile.

7 Discussion

The claim is structural, and the numbers only illustrate it. The argument is not "our method scores higher." It is that text is the wrong container for half of what an agent should remember about a person, and that a parametric in-model bank is the right one. The empirical results matter because they show the alternative container is lossless and *free*: it reproduces the encoder's recall exactly, on any frozen model, with no training, while turning the recalled perception into a token the model can act on. The contribution is the container, not a recall score; the score is the encoder's, and the container is what an external index cannot provide.

This is additive infrastructure, not a replacement. The perceptual memory is designed to bolt onto the text memory that agents already have [Chhikara et al., 2025, Wang et al., 2024, Packer et al., 2023, Team, 2024], and the non-regression result is what makes that safe: text recall is preserved exactly, and the perceptual bank activates only at perceptual moments. As long-term memory benchmarks like LongMemEval [Wu et al., 2024] grow to include perceptual content, the gap this paper argues for becomes directly measurable.

The captionable half is going parametric too. This paper is one half of a larger bet: a personalized agent should keep its memory *inside* the model, not in an external index. A companion line makes the same bet for the *captionable* half. *User as Engram* [Li, 2026c] writes per-user *facts* into a hash-keyed parametric memory, showing that the obvious alternative (a per-user LoRA adapter) is *reasoning-negative*: it memorizes facts but degrades the reasoning that uses them, because a LoRA edit is global where a row insertion is local. *User as Code* [Li, 2026b] pushes the same half into executable typed state, and *Programmable KV Cache* [Li, 2026a] keeps it as editable, composable notes in the attention cache rather than in weights or an external index. Our perceptual counterpart shares that thesis with a different mechanism, continuous cross-attention over encoder banks rather than hash-keyed rows. The two halves compose into one memory that holds both what the user said and what they are like.

What a latent memory can and cannot hold. The split between the two stores is not a convention. It follows from two capacity laws that we measured by compressing content into k

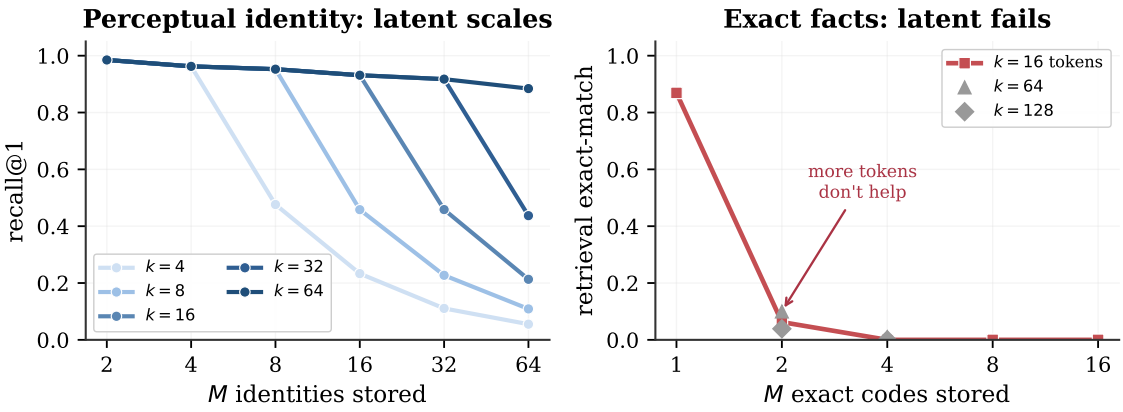


Figure 6. Two capacity laws for memory compressed into k soft tokens read by a frozen language model, the mechanism behind the router. **Left:** perceptual identity is capacity-limited. Compressing M faces into k prototype slots and recognising a cross-condition query gives recall $\approx \min(1, k/M)$, so a slot per identity recognises everyone and more slots hold more people (voices behave identically). **Right:** exact factual content is binding-limited. A latent holds one exact code (0.87) but multi-code retrieval collapses at two (0.06) and adding latent tokens does not rescue it. Recognition scales with the latent budget. Exact recall does not, so it belongs in a text store.

soft tokens read by a frozen language model (Figure 6). Perceptual identity is capacity-limited and degrades gracefully. Compressing M registered identities into k prototype slots and recognising a cross-condition query keeps a k/M fraction of them distinguishable, on faces, voices, acoustic scenes, painting styles, and tones of voice alike (Appendix Table 5). One slot per identity recognises as many as the encoder can, and fewer slots lose identities in proportion. The general law is recall $\approx \min(1, k/M) \cdot C(M)$, where $C(M)$ is the encoder's own recall at M registrations: near 1 for strong encoders (face, voice), lower for the weaker style and tone encoders, so the slot-compression factor $\min(1, k/M)$ is what is universal and the ceiling is the encoder's. This compression law is not an artifact of k-means: a compressor whose k slots we instead learn by gradient descent lands on the same $\min(1, k/M)$ curve, because k hard slots can keep at most k of M identities separable (a pigeonhole bound). The only way past it is to attach a per-identity soft code over the slots, which restores accuracy but costs storage linear in M, at which point one may as well keep the M keys. Exact factual content behaves in the opposite way. A latent holds one exact code and reads it back (0.87 for a six-character code), but storing several and retrieving one by name collapses at once (0.06 at two codes, near zero beyond), and adding latent tokens does not help, with recall pinned at the floor from sixteen to a hundred and twenty-eight tokens. Recognition is a capacity problem, where a latent buys one identity per slot. Exact recall is a binding problem, where a latent cannot match a key to its value at any budget. That is why the captionable half wants a text store, which performs that lookup exactly, and the perceptual half wants a parametric bank, which scales with its slots.

Limitations. (i) The memory cannot exceed the encoder. Its recall *is* the encoder's, so at very large memories it declines exactly as the encoder's cosine does (0.77 at a thousand faces), and no in-model trick recovers more. If a task needs better perceptual recall, the lever is a better encoder, not this memory. (ii) We have measured real cross-condition data to roughly two thousand identities. The constant-time behaviour is confirmed to ten thousand, but recall at that scale awaits data. (iii) Inside a VLM, native vision tokens fail as keys and an external, modality-specific encoder is required. (iv) The one per-model constant (read-strength gain) is

set by inspection; untied-embedding models need a larger value, which we have not automated. (v) The closest prior systems (MyVLM [Alaluf et al., 2024], Online-PVLM [Bai et al., 2025]) have released neither code nor checkpoints. We compare against charitable reimplementations of their core mechanisms (Appendix A, Table 7); a faithful head-to-head against their full systems awaits release.

8 Conclusion

A photograph and a written description of a face are not two encodings of the same thing. The description has already thrown away what lets you recognize the person. Agent memory today keeps only the description. We have argued it must keep the photograph too. The perception is stored in its native modality, as a content-addressable row in a bank on a frozen language model, not flattened into words it cannot survive. The two memories are complementary: text for what the user *said*, parametric multimodal memory for what the user *is like*. An agent with both strictly dominates one with only text, recalling the captionable half exactly and holding the perceptual half at all.

Empirically, keeping the perception continuous reproduces the encoder's recall exactly, on every frozen model we tried, with no training and a single constant as the only knob. The memory does not beat the encoder; it gives the encoder's recall a home inside the model. That is what a retrieval index cannot copy: the recalled perception is a token the model conditions on, registered in constant time, composing with text recall, cheap enough to update after every conversation and portable to any frozen model out of the box. That is where a face stops being a sentence, and starts being a memory.

Acknowledgements

AI tools including Pine Copilot, Claude Code with Claude Opus 4.8 were used during this research. We thank BSQL Networking for hosting the RTX PRO 6000 GPU.

A Detailed results

This appendix collects the per-cell numbers behind Sections 5–6. Recall is top-1 with the argmax restricted to registered markers.

Paired evaluation protocol. A comparison between the memory and retrieval requires two controls: both methods score on *identical* registrations and queries, and each target identity occupies a randomly chosen bank slot. The second control matters because a fixed target slot lets the constant marker token in that slot accrue a baseline output-logit, which inflates recall independently of the query. We score both methods on the same twenty draws with the target slot randomised, and report mean$\pm$std. Table 3 gives the result. The memory ties retrieval on random banks, since with a sharp read it computes the same argmax over encoder cosine, and on banks of the nineteen most-confusable look-alikes it trails by a few points: its similarity-weighted blend of marker values carries no per-identity signal beyond the cosine, so it cannot separate confusable identities the encoder already conflates.

Table 3. Paired evaluation: the memory and retrieval scored on identical registrations and queries across 20 draws, with the target slot randomised. The memory ties retrieval on random banks and trails it by a few points on look-alike banks. Recall is the encoder's throughout.

Regime	Cell	Retrieval	AttMem (mean±std)	Δ
random	Face, N=10	0.948	0.942 ± 0.035	−0.6pp
random	Style, N=5	0.473	0.476 ± 0.116	+0.2pp
random	Style, N=10	0.428	0.357 ± 0.081	−7.2pp
adversarial	Face, K=19	0.853	0.773 ± 0.008	−8.0pp

Universality grid. Table 4 runs the training-free read across five model architectures and all five modalities, scoring each cell against the encoder with the same paired protocol (20 draws). Every one of the twenty-five cells reproduces the encoder's recall to within one point (worst case 0.010, most cells ≤ 0.003). Tied-embedding models use the default read-strength gain; untied and hybrid models use a single larger value (256), set once per model and held fixed across modalities. The read therefore inherits the encoder's recall regardless of the host model's family, embedding scheme, scale, or even whether it is a transformer.

Table 4. Universality grid: maximum $|\Delta|$ between the training-free memory and the encoder over $N \in \{5, 10, 20\}$ (paired, 20 draws), for five architectures × five modalities. Every cell is within one point of the encoder. "gain" is the single per-model read-strength constant. Encoder recall@1 at N=10: Face 0.95, Speaker 0.99, Acoustic 0.86, Tone 0.52, Style 0.43.

Model	Emb.	gain	Face	Speaker	Acoustic	Tone	Style
Qwen2.5-7B	tied	64	0.001	0.000	0.002	0.003	0.001
Phi-3.5-mini	tied	64	0.001	0.000	0.000	0.002	0.002
DeepSeek-Llama-8B	untied	256	0.000	0.000	0.002	0.002	0.000
Mistral-7B	untied	256	0.000	0.000	0.003	0.010	0.000
Granite-4.0 (Mamba)	hybrid	256	0.003	0.000	0.003	0.001	0.000

Capacity law across modalities. Table 5 runs the perceptual capacity probe (compress M registered identities into k prototype slots, recognise a cross-condition query, 5 seeds) on all five modalities. The slot-compression factor $\min(1, k/M)$ fits every modality once normalised by the one-slot-per-identity ceiling $C(M)$: the normalised error is ≤ 0.012 on the strong-encoder modalities and ≤ 0.13 on the weaker style and tone encoders. The ceiling $C(M)$ itself is the encoder's recall and varies accordingly, from near 1 on voice to 0.26–0.36 at M=32 on style and tone. Recognition is capacity-limited and graceful in every modality; only the ceiling moves.

Learned-metric baseline (could a re-encoding do better?). A natural question is whether *any* re-encoding of the same features beats raw cosine, even if our read does not. We fit a similarity on the same encoder features and the same identities (the 50/50 split, seed 42) and score it with the identical protocol. Two closed-form metrics suffice: regularised LDA and within-class whitening. Table 6 shows they land on raw cosine at small memory and *below* it past a few hundred identities. So there is no discrimination left to extract from these features: the encoder's cosine is already the best ruler, which is why the memory's inheriting it exactly

Table 5. Perceptual capacity law per modality. $C(M)$ is the one-slot-per-identity ceiling (recall at $k{=}M$), i.e. the encoder's recall at M. "fit" is the mean $|\,\text{recall}/C(M) - \min(1, k/M)\,|$ over the $k{<}M$ cells: the slot-compression factor is universal; the ceiling is the encoder's.

Modality	$C(2)$	$C(8)$	$C(32)$	**fit to** $\min(1, k/M)$
Face (ArcFace)	0.98	0.95	0.92	0.002
Voice (ECAPA)	1.00	1.00	0.99	0.001
Acoustic (AST)	0.95	0.87	0.76	0.012
Style (CLIP)	0.76	0.46	0.26	0.070
Tone (wav2vec)	0.94	0.67	0.36	0.125

is the ceiling, not a shortfall. A contrastive linear head we also trained underperformed both closed-form metrics and is omitted as a weak instantiation.

Table 6. Learned-metric baseline, recall@1, same encoder and same split as AttMem. A learned metric (LDA on faces, whitening on style) matches AttMem's small-N lift over raw cosine; at large N raw cosine wins for all. Style AttMem here is the seed-42 single run (the multi-seed mean is 0.64 at $N{=}5$); the comparison is within seed.

Modality	N	**Raw cosine**	**LDA**	**Whitening**	**AttMem**
Face (ArcFace)	5	0.933	**1.000**	0.933	0.933
Face (ArcFace)	10	0.933	0.967	0.933	**1.000**
Face (ArcFace)	300	0.734	0.667	**0.744**	0.641
Face (ArcFace)	1000	0.767	0.724	**0.776**	—
Style (CLIP-mid)	5	0.400	0.200	**0.600**	0.467
Style (CLIP-mid)	10	0.400	0.333	0.367	**0.467**

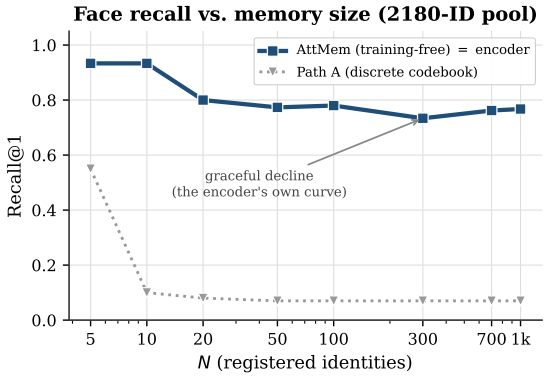
Per-concept methods: a charitable head-to-head. The personalized-VLM line (MyVLM, Yo'LLaVA, Online-PVLM) registers a concept by *learning* something per concept (a classifier head, a soft token, a projection). Their code and checkpoints are unreleased, so we reimplement each method's core mechanism charitably on the cross-condition face task (ArcFace keys, same split). Table 7 reports accuracy and registration cost. Two findings. First, a per-concept linear classifier (MyVLM's mechanism) is statistically indistinguishable from raw cosine at every memory size, because with one registration embedding per identity the trained head collapses to the embedding itself. It buys no accuracy over retrieval, and it costs per-identity gradient descent that grows to 4.5 s at a thousand identities, against our $\mathcal{O}(1)$ tensor append. Second, a learned projection trained with supervised contrastive loss (Online-PVLM's mechanism) *underperforms* raw cosine by 15 to 28 points on this task. We read this as a caution that a learned re-encoding is not free: tuned on identity-disjoint data, it can distort the very cross-condition geometry it was meant to sharpen. No method here, ours included, beats raw cosine on accuracy at scale; the parametric memory's case rests on cost and in-model behaviour, not recall.

Scaling and the codebook comparison. Figure 7a traces recall against memory size on the face pool. The training-free memory tracks the encoder's own curve (it *is* that curve), declining gracefully from near-ceiling at ten identities to 0.77 at a thousand, while the discrete-codebook

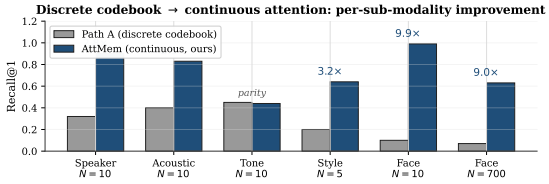
Table 7. Charitable reimplementations of per-concept methods on cross-condition face recall (ArcFace, recall@1, single eval draw). MyVLM's per-concept classifier ties raw cosine but its registration cost grows with the memory; Online-PVLM's learned projection underperforms raw cosine. AttMem leads only at small N and trails at scale (the encoder-ceiling inversion).

N	Raw cosine	MyVLM	Online-PVLM	AttMem	MyVLM reg. cost
5	0.933	0.933	0.800	0.933	0.009 s
10	1.000	0.967	0.750	0.992	0.019 s
50	0.767	0.753	0.675	0.733	0.178 s
100	0.780	0.770	—	0.742	0.528 s
300	0.728	0.725	—	0.637	1.39 s
1000	0.776	0.778	—	0.594	4.52 s

predecessor (PATH A) flatlines at ∼0.07 regardless of codebook size or budget. Figure 7b makes the same point across all five sub-modalities: replacing a discrete codebook with continuous attention lifts recall 2–10×, because a categorical bottleneck discards the signal the encoder preserved. This is the one place a design choice in the read matters; everything else inherits the encoder.



(a) scaling on the face pool



(b) codebook vs. continuous attention

Figure 7. Left: the continuous memory degrades gracefully with memory size (tracking the encoder ceiling), while the codebook predecessor never gets off the floor. Right: continuous attention beats the discrete codebook 2–10× on every sub-modality.

Vision-language model, key/value orthogonality. Table 8 gives the numbers behind the structural rule of Section 6. Each memory row is compared to retrieval *in its own key space*. An external ArcFace key (orthogonal to the model's hidden space) reproduces the win. The VLM's native vision tokens (already in hidden space) only tie native-token retrieval and stay far below the ArcFace ceiling throughout, so they are the wrong key regardless of method.

B Anatomy of a single recall

It helps to watch one recall end to end. Suppose the agent has met ten people and wants to register an eleventh from a single photo. Registration is one row appended to the face bank (Listing 1). The encoder turns the photo into a 512-dimensional ArcFace embedding, the *key*.

Table 8. Parametric memory inside Qwen2.5-VL-3B: same frozen model, only the key encoder changes. Recall at three memory sizes.

Configuration	N=10	N=100	N=1000
Retrieval over ArcFace embeddings (encoder ceiling)	0.933	0.780	0.767
AttMem + external ArcFace keys (orthogonal to hidden space)	**1.000**	0.710	0.494
Retrieval over native vision tokens	0.40	0.35	—
AttMem + native vision tokens (co-located with hidden space)	0.40	0.11	—

The model's own embedding for a freshly assigned marker token becomes the *value*, a vector of the model's hidden width (2048 for Qwen2.5-3B). There is no gradient step and no fine-tuning. The row is the memory.

```
# Register identity #11 from a single photo -- no training:
k = l2_normalize(arcface(photo))        # key:   R^512  (encoder space)
m = assign_marker_token()               # e.g. "<id_11>"
v = model.input_embedding[m]            # value: R^2048 (model's own space)
bank.K = torch.cat([bank.K, k[None]])   # O(1) append
bank.V = torch.cat([bank.V, v[None]])
```

Listing 1. One registered identity is one row. The key is the encoder's view of the perception; the value is the model's own vector for the marker token that names it. Adding the row is a single tensor append.

At recall, a new photo of one of the eleven arrives under different lighting and a year older. Its ArcFace embedding becomes the query $\boldsymbol{q}$. Just before its output head, the model scores $\boldsymbol{q}$ against all eleven keys, softmaxes into attention weights $\boldsymbol{w}$, and pulls back the weighted blend of values $\boldsymbol{r} = \boldsymbol{w}^\top \boldsymbol{V}$. Because each value is the model's own vector for a marker token, adding $g\, W_o \boldsymbol{r}$ to the hidden state lands as a clean boost on the right marker's logit. The model's next token *is* the remembered identity. The whole step is a few matrix multiplies dwarfed by the model's forward pass.

The 0.98-vs-0.46 diagonal gap of Section 5 (Figure 5) is measured on exactly this probe: a held-out ten-identity bank, comparing the sharp attention weights against the raw encoder cosine they are computed from. The attention concentrates the cosine onto its top match, which is why the read recovers the encoder's nearest neighbour as a single decisive marker rather than a diffuse blend.

C Why the quantized predecessor capped at 7%

Before continuous attention we spent sixteen development cycles on PATH A: a discrete codebook that snaps each perception to one of K learned codes and remembers the code. It is the natural "compress the perception into a symbol" design, and it is a learned cousin of captioning, which is why its failure is instructive rather than embarrassing.

No setting broke it. Across codebook sizes $K \in \{128, 256, 512, 1024\}$, even after *100,000* steps of continual pretraining on the expanded face pool, top-1 recall at 300 identities stays pinned between 0.057 and 0.070 while embedding retrieval over the same encoder reaches 0.73. That is a roughly 10× gap that does not move with K (Table 9). Swapping ArcFace R50 for AntelopeV2 R100 trained on 360K identities did not help either.

Table 9. PATH A after 100K-step continual pretraining on the 2180-identity face pool, at 300 registered identities. Recall is pinned near 0.07 regardless of codebook size, even when the gate routes to the correct code about half the time.

Codebook size K	128	256	512	1024
PATH A recall@1	0.057	0.064	0.070	0.070
gate routes to correct code	0.43	0.51	0.42	0.54
embedding retrieval (same encoder)		0.73		

The reason is a vice the codebook cannot escape, and the diagnostics name it exactly. Two pressures pull in opposite directions as K grows. A *small* codebook packs many people into each cell: at K=16, distinct identities collide in the same code 8.7% of the time, and once two people share a code they are indistinguishable forever after. A *large* codebook fixes that, with inter-identity collisions falling to 2.4% at K=128, but it shatters each identity across cells. The rate at which two cross-condition photos of the *same* person land in the same code drops from 0.33 to 0.20, so the query no longer routes to where the registration was stored. Net recall is squeezed from both sides and never escapes $\sim$0.07. Even when the gate does route a query to the right code (about half the time, Table 9), every other identity sharing that cell is an equally good answer, so the final pick is near-random.

This is the same information loss as captioning, learned instead of written: any categorical bottleneck, whether a word or a code, throws away the continuous signal the encoder worked to preserve, and no amount of compute downstream can recover it. Continuous attention over the raw embedding never quantizes, and never pays this tax. PATH A is in the paper because its failure is the cleanest possible argument for the design that replaced it.

D Reproducibility

The system is about 200 lines of new code over a frozen-model `transformers` stack. The read is training-free, so the eval entry point runs with `n_steps=0` and two constants set by environment variables (`ATTMEM_INV_TEMP`, `ATTMEM_OUT_GAIN`). Every run logs to `results/`. The paired protocol scores the memory and retrieval on identical draws (`ATTMEM_PAIRED_NS`) with the target slot randomised. An eval is $\sim$2–5 minutes on an H100-class GPU; there is no training step.

```
# Training-free paired eval (recall == encoder), all five modalities
export ATTMEM_INV_TEMP=500 ATTMEM_OUT_GAIN=64 ATTMEM_PAIRED_NS=5,10,50
for mode in v-xc-id-xxxl a-xr-id a-scn a-para v-sty-clip; do
  python3 attmem_train_and_eval.py $mode 0 42
done

# Universality across frozen model families (untied models need a larger gain)
for m in Qwen/Qwen2.5-7B-Instruct Qwen/Qwen3-8B microsoft/Phi-3.5-mini-instruct \
         deepseek-ai/DeepSeek-R1-Distill-Llama-8B ibm-granite/granite-4.0-h-tiny; do
  ATTMEM_MODEL_ID="$m" python3 attmem_train_and_eval.py v-xc-id-xxxl 0 42
done
ATTMEM_OUT_GAIN=256 ATTMEM_MODEL_ID="mistralai/Mistral-7B-Instruct-v0.3" \
  python3 attmem_train_and_eval.py v-xc-id-xxxl 0 42
```

```
# Capacity laws, VLM key geometry, latency, text non-regression
python3 latentmem/set_memory.py; python3 latentmem/code_memory.py
python3 attmem_vl_eval.py
python3 attmem_latency_benchmark.py
python3 attmem_propositional_control.py
```

References

Yuval Alaluf, Elad Richardson, Sergey Tulyakov, Kfir Aberman, and Daniel Cohen-Or. MyVLM: Personalizing VLMs for user-specific queries. In *European Conference on Computer Vision (ECCV)*, 2024.

Ruichuan An, Sihan Yang, Ming Lu, Kai Zeng, Yulin Luo, Ying Chen, Jiajun Yong, Huanqia Liang, Qi Huang, and Shanghang Zhang. MC-LLaVA: Multi-concept personalized vision-language model. *arXiv preprint arXiv:2411.11706*, 2024.

Alexei Baevski, Henry Zhou, Abdelrahman Mohamed, and Michael Auli. wav2vec 2.0: A framework for self-supervised learning of speech representations. *Advances in Neural Information Processing Systems*, 2020.

Huiyu Bai, Runze Wang, Zhuoyun Du, Yiyang Zhao, Fengji Zhang, Haoyu Chen, Xiaoyong Zhu, Bo Zheng, and Xuejiao Zhao. Online-PVLM: Advancing personalized VLMs with online concept learning. *arXiv preprint arXiv:2511.20056*, 2025.

Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.

Jiankang Deng, Jia Guo, Niannan Xue, and Stefanos Zafeiriou. ArcFace: Additive angular margin loss for deep face recognition. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

Brecht Desplanques, Jenthe Thienpondt, and Kris Demuynck. ECAPA-TDNN: Emphasized channel attention, propagation and aggregation in TDNN based speaker verification. In *INTERSPEECH*, 2020.

Yuan Gong, Yu-An Chung, and James Glass. AST: Audio spectrogram transformer. In *INTERSPEECH*, 2021.

Haoran Hao, Jiaming Han, Changsheng Li, Yu-Feng Li, and Xiangyu Yue. RAP: Retrieval-augmented personalization for multimodal large language models. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

Gary B Huang, Marwan Mattar, Tamara Berg, and Eric Learned-Miller. Labeled faces in the wild: A database for studying face recognition in unconstrained environments. In *Workshop on Faces in Real-Life Images*, 2008.

Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. In *International Conference on Learning Representations (ICLR)*, 2020.

Bojie Li. Models take notes at prefill: KV cache can be editable and composable. *arXiv preprint arXiv:2606.17107*, 2026a.

Bojie Li. User as code: Executable memory for personalized agents. *arXiv preprint arXiv:2606.16707*, 2026b.

Bojie Li. User as engram: Internalizing per-user memory as local parametric edits. *arXiv preprint arXiv:2606.19172*, 2026c.

Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *International Conference on Machine Learning (ICML)*, 2023.

Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Steven R Livingstone and Frank A Russo. The Ryerson audio-visual database of emotional speech and song (RAVDESS). volume 13, 2018.

Lin Long, Yichen He, Wentao Ye, Yiyuan Pan, Yuan Lin, Hang Li, Junbo Zhao, and Wei Li. Seeing, listening, remembering, and reasoning: A multimodal agent with long-term memory (M3-Agent). *arXiv preprint arXiv:2508.09736*, 2025.

Stylianos Moschoglou, Athanasios Papaioannou, Christos Sagonas, Jiankang Deng, Irene Kotsia, and Stefanos Zafeiriou. AgeDB: The first manually collected, in-the-wild age database. In *CVPR Workshops*, 2017.

Thao Nguyen, Haotian Liu, Yuheng Li, Mu Yi, Jiarui Mu, and Yong Jae Lee. Yo'LLaVA: Your personalized language and vision assistant. *arXiv preprint arXiv:2406.09400*, 2024.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G Patil, Ion Stoica, and Joseph E Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

Vassil Panayotov, Guoguo Chen, Daniel Povey, and Sanjeev Khudanpur. LibriSpeech: An ASR corpus based on public domain audio books. 2015.

Karol J Piczak. ESC: Dataset for environmental sound classification. In *ACM Multimedia*, 2015.

Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*, 2021.

Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. *arXiv preprint arXiv:2212.04356*, 2022.

Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-Networks. In *Empirical Methods in Natural Language Processing (EMNLP)*, 2019.

Letta Team. Letta (formerly MemGPT): Stateful LLM agents with long-term memory. *arXiv preprint arXiv:2310.08560*, 2024. Software framework.

Yu Wang, Dmitry Krotov, Yuanzhe Hu, Yifan Gao, Wangchunshu Zhou, Julian McAuley, Dan Gutfreund, Rogerio Feris, and Zexue He. MemoryLLM: Towards self-updatable large language models. *arXiv preprint arXiv:2402.04624*, 2024.

WikiArt. WikiArt: Visual art encyclopedia. 2010. https://www.wikiart.org.

Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2024.

Yuhuai Wu, Markus N Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *International Conference on Learning Representations (ICLR)*, 2022.